

# NEORV32 RISC-V Processor

## Complete Simulation Guide on Linux

*From Zero to Hello World in Simulation*

|                                                          |                                                  |
|----------------------------------------------------------|--------------------------------------------------|
| <b>Target Board</b><br>Nexys 4 Legacy (Artix-7 XC7A100T) | <b>OS</b><br>Ubuntu 24.04 LTS (Linux)            |
| <b>Simulator</b><br>GHDL + Icarus Verilog                | <b>Compiler</b><br>xPack riscv-none-elf-gcc 14.x |

*April 2026*

## Overview

This guide walks you through simulating the NEORV32 RISC-V processor on your laptop running Ubuntu Linux, before moving to real FPGA hardware. You will go from a fresh Ubuntu install to running a C 'Hello World' program on a simulated RISC-V CPU.

The guide covers two simulation paths:

- Path A — GHDL simulation (VHDL testbench, runs hello\_world C program)
- Path B — Icarus Verilog simulation (converted Verilog wrapper, runs bootloader)

### **i** NOTE

This guide was written based on real troubleshooting experience. Every error encountered and its fix is documented. Follow the steps exactly in order.

## Tools Summary

| Tool                     | Purpose                            | Install Command                                |
|--------------------------|------------------------------------|------------------------------------------------|
| GHDL 4.x                 | VHDL simulator / Verilog converter | <code>sudo apt install ghdl</code>             |
| Icarus Verilog           | Verilog simulator                  | <code>sudo apt install iverilog</code>         |
| GTKWave                  | Waveform viewer                    | <code>sudo apt install gtkwave</code>          |
| xPack riscv-none-elf-gcc | RISC-V C compiler                  | Download from GitHub releases                  |
| make, git, python3       | Build tools                        | <code>sudo apt install make git python3</code> |

## Repository Structure

You will clone two separate repositories:

| Repository                             | Purpose                                             |
|----------------------------------------|-----------------------------------------------------|
| <code>stnolting/neorv32</code>         | Main repo — VHDL source, C software, GHDL testbench |
| <code>stnolting/neorv32-verilog</code> | Conversion repo — VHDL→Verilog + Icarus testbench   |

## Phase 1 — Install All Tools

### 1.1 Install system packages

```
sudo apt update
sudo apt install -y ghdl ghdl-mcode gtkwave iverilog make git python3
```

### 1.2 Verify GHDL and Icarus

```
ghdl --version
iverilog -V
```

#### ⚠ WARNING

You must see GHDL 4.x or later. Older versions may not support VHDL-2008 which NEORV32 requires.

### 1.3 Install xPack RISC-V GCC toolchain

The Ubuntu package `riscv-none-elf-gcc` does NOT include full newlib headers (`inttypes.h` etc.) needed by NEORV32. Use the xPack prebuilt toolchain instead.

```
cd ~
wget https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases/download/v14.2.0-3/xpack-riscv-none-elf-gcc-14.2.0-3-linux-x64.tar.gz
mkdir -p ~/riscv-xpack
tar -xzf xpack-riscv-none-elf-gcc-14.2.0-3-linux-x64.tar.gz -C ~/riscv-xpack --strip-components=1
echo 'export PATH=~/riscv-xpack/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

#### ℹ NOTE

If you use `zsh` instead of `bash`, replace `~/.bashrc` with `~/.zshrc` in the `echo` command.

### 1.4 Verify the compiler

```
riscv-none-elf-gcc --version
```

Expected output: riscv-none-elf-gcc (xPack GNU RISC-V Embedded GCC x86\_64) 14.2.0

✓ **CHECKPOINT**

riscv-none-elf-gcc --version shows xPack 14.x and ghdl --version shows 4.x

## Phase 2 — Clone the Repositories

### 2.1 Create a project folder and clone both repos

```
mkdir -p ~/ITU
cd ~/ITU

# Clone main NEORV32 repo
git clone --recursive https://github.com/stnolting/neorv32.git

# Clone the Verilog conversion repo
git clone --recursive https://github.com/stnolting/neorv32-verilog.git
```

#### ⚠ WARNING

The `--recursive` flag is CRITICAL. Without it, the `neorv32-verilog` repo will be missing the `neorv32` submodule and conversion will fail.

### 2.2 Verify the folder structure

```
ls ~/ITU/neorv32/
# Expected: rtl/  sim/  sw/  docs/  ...

ls ~/ITU/neorv32-verilog/
# Expected: src/  sim/  Makefile  README.md
```

#### ✓ CHECKPOINT

Both folders exist and `neorv32/rtl/core/` contains `.vhd` files

## Phase 3 — Path B: Verilog Conversion + Icarus Simulation

This path converts NEORV32 from VHDL to Verilog using GHDL, then simulates with Icarus Verilog. It runs the default bootloader and verifies the processor boots correctly.

### 3.1 Fix the testbench timeout

The default testbench has a 15ms timeout which is too short. Edit it:

```
nano ~/ITU/neorv32-verilog/sim/testbench.v
```

Find this line (use Ctrl+W to search):

```
#15_000_000;
```

Change it to:

```
#2_000_000_000;
```

Save with Ctrl+O, Enter, then exit with Ctrl+X.

### 3.2 Convert VHDL to Verilog and simulate

```
cd ~/ITU/neorv32-verilog
make check
make clean && make SIMULATOR=iverilog sim
```

The make clean is important — without it Make may skip rebuilding if old output files exist.

Expected output after a few minutes:

```
neorv32-verilog verification testbench

NEORV32
Simulation successful!
$finish called at 81863550 (100ps)
```

✓ CHECKPOINT

You see 'Simulation successful!' — the bootloader printed NEORV32 over simulated UART

**i NOTE**

The conversion step (make convert) runs GHDL synthesis to produce `src/neorv32_verilog_wrapper.v` — a single large Verilog file (~5000+ lines) that is the entire processor. You can open and read this file as it is plain Verilog.

## Phase 4 — Path A: GHDL Simulation with Hello World C Program

This path compiles a real C program, loads it into the simulated processor's IMEM, and runs the full GHDL testbench. You will see Hello World printed from your C code running on the simulated RISC-V CPU.

### 4.1 Edit the testbench to reduce simulation overhead

The default testbench enables dual-core, caches, and instruction tracer which makes GHDL very slow. Disable them:

```
nano ~/ITU/neorv32/sim/neorv32_tb.vhd
```

Use Ctrl+W to search and find each line. Change these 4 values from true to false:

```
-- Find and change these lines:
DUAL_CORE_EN      : boolean      := false;
ICACHE_EN         : boolean      := false;
DCACHE_EN         : boolean      := false;
TRACE_LOG_EN      : boolean      := false;
```

Save with Ctrl+O, Enter, then Ctrl+X.

#### **NOTE**

This makes simulation 8-10x faster by removing dual-core, cache logic, and disk logging of every instruction.

### 4.2 Compile the hello\_world C program and install to IMEM

```
cd ~/ITU/neorv32/sw/example/hello_world
make MARCH=rv32imac_zicsr_zifencei USER_FLAGS+=-DUART0_SIM_MODE clean_all install
```

What each flag does:

- MARCH=rv32imac\_zicsr\_zifencei — sets the RISC-V ISA extensions for the compiler
- USER\_FLAGS+=-DUART0\_SIM\_MODE — redirects UART output to simulation log instead of real UART pin (otherwise output would never appear in simulation)
- clean\_all — removes old build artifacts
- install — copies the compiled memory image to rtl/core/neorv32\_imem\_image.vhd



Expected output includes:

```
Memory utilization:
  text    data    bss    dec    hex filename
  4440     0    256   4696   1258 main.elf
Generating neorv32_imem_image.vhd
Executable (VHD): 4440 bytes
Installing application image to ../../../../rtl/core/neorv32_imem_image.vhd
```

#### **I NOTE**

In older NEORV32 versions the installed file was called neorv32\_application\_image.vhd. In v1.12+ it is neorv32\_imem\_image.vhd. Both work the same way.

### **4.3 Verify the image file was created**

```
ls ~/ITU/neorv32/rtl/core/neorv32_imem_image.vhd
```

#### **X ERROR FIX**

If this file does not exist, the install step did not complete. Re-run step 4.2 and check for errors.

### **4.4 Delete old GHDL build cache and run simulation**

```
rm -rf ~/ITU/neorv32/sim/build
cd ~/ITU/neorv32/sim
sh ghdl.sh --stop-time=100ms
```

Open a second terminal and watch for output:

```
watch -n1 'cat ~/ITU/neorv32/sim/tb.uart0_rx.log'
```

After a few minutes you should see:

```
##          ##          #####          #####          #####          ...          ##          ##          ##          ##
##          ##          #####          #####          #####          ...          ##          #####          #####
...
Hello world! :)
```

#### **I NOTE**

The simulation prints to tb.uart0\_rx.log, not to the terminal directly. Always check that file for UART output.

✓ **CHECKPOINT**

tb.uart0\_rx.log contains the NEORV32 ASCII logo and 'Hello world! :)' — your C program ran on the simulated RISC-V CPU

## Troubleshooting Reference

| Error                                 | Cause                                       | Fix                                                                                   |
|---------------------------------------|---------------------------------------------|---------------------------------------------------------------------------------------|
| make: No rule to make target 'sim'    | Wrong directory                             | cd ~/ITU/neorv32-verilog<br>first                                                     |
| make: 'sim' is up to date             | Old output file exists                      | Run make clean first, then<br>make sim                                                |
| ISA string must begin with rv32       | MARCH variable has duplicate<br>prefix      | Use<br>MARCH=rv32imac_zicsr_zifencei<br>(no -march= prefix)                           |
| fatal error: inttypes.h not found     | Ubuntu riscv-none-elf-gcc<br>missing newlib | Install xPack toolchain (Phase 1,<br>Step 1.3)                                        |
| unrecognized opcode 'fence.i'         | Missing zifencei ISA extension              | Add _zifencei to MARCH flag                                                           |
| simulation stopped at 10ms, no output | Default stop time too short                 | Use GHDL_RUN_FLAGS="--stop-<br>time=100ms"                                            |
| tb.uart0_rx.log is empty              | neorv32_imem_image.vhd<br>missing           | Run make install in hello_world<br>folder first                                       |
| vvp runs forever, no output           | Testbench has no stop time                  | Edit testbench.v, change<br>#15_000_000 to #2_000_000_000                             |
| GHDL sim extremely slow (hours)       | Dual-core + caches + tracer<br>enabled      | Set DUAL_CORE_EN, ICACHE_EN,<br>DCACHE_EN, TRACE_LOG_EN to<br>false in neorv32_tb.vhd |

## What's Next — FPGA Implementation

Now that simulation is working, the next phase is synthesizing NEORV32 for the Nexys 4 board in Vivado:

- Install Vivado WebPACK (free) — select Artix-7 support during installation
- Create a new Vivado project targeting xc7a100tcsg324-1 (Nexys 4 Artix-7)
- Add all VHDL files from rtl/core/ to the project under a library named neorv32
- Use rtl/test\_setups/neorv32\_test\_setup\_bootloader.vhd as the top module
- Add the Nexys 4 constraints file (.xdc) mapping: 100MHz clock pin, CPU reset button, USB-UART TX/RX pins
- Run Synthesis → Implementation → Generate Bitstream
- Open Hardware Manager, program the .bit file to the board
- Open PuTTY or minicom at 115200 baud — you will see the NEORV32 bootloader prompt
- Upload neorv32\_exe.bin over UART to run your C program on real hardware

**i NOTE**

The Nexys 4 Legacy board uses Cellular RAM instead of DDR2. This does not affect NEORV32 since it uses internal BRAM for IMEM and DMEM. No external RAM connection is needed for basic operation.